

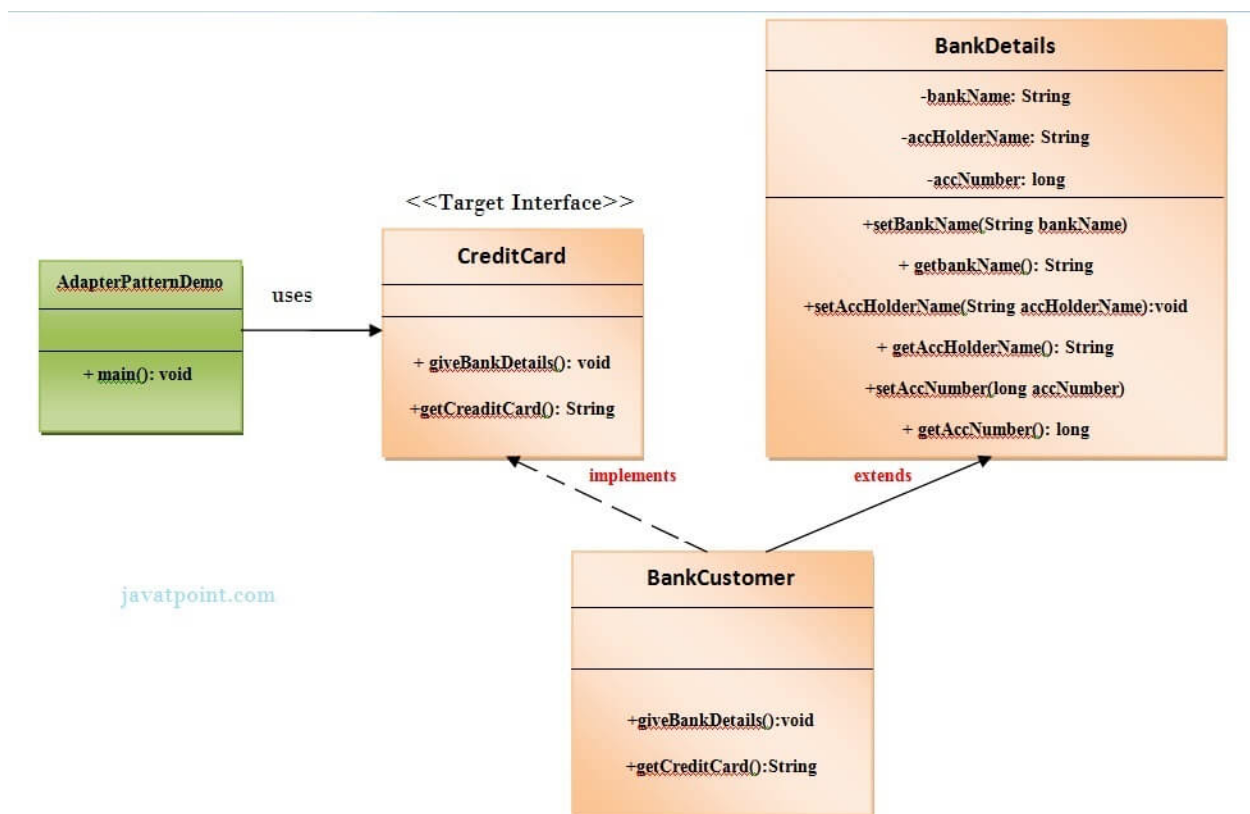
ADVANCE PROGRAMMING

TUTORIAL

Design Patterns:

Design patterns are ways of solving problems that occur while designing systems. Of course, not all problems are anticipatable in advance, but creating a set of solutions for commonly encountered problems holds us in good stead. In this tutorial, we will cover a problem based on the Adapter design pattern covered in class.

In this question, you are given a UML diagram, consisting of classes and interfaces. The UML diagram is given below:



Here, AdapterPatternDemo is the main class, and wishes to utilize the functionalities provided by the Credit Card interface. However, the BankDetails class that has been provided has its own methods, similar in functionality to what the user wants, but not exactly. Modifications in these methods are required.

Therefore, for this purpose, an adapter class called BankCustomer is created, and it overrides the methods in the Credit Card interface. The BankCustomer class also

extends from the BankDetails class, so that in its functions, it can customize the functionalities of the BankDetails class as per user requirements.

The functionalities of the methods are as follows:

- 1) The BankDetails class defines only getters and setters, so their purpose is straightforward.
- 2) In the giveBankDetails() method, you basically have to take user input, and use them to set the values of the attributes of the BankDetails class.
- 3) In the getCreditCard() method, you need to use the getter methods of the BankDetails class, and print them along with the message that “this account is valid and authenticated for issuing the crediting card”.
- 4) In the main class, just create an object of the BankCustomer class, and use that object to call the 2 methods.

SOLUTION:

```
import java.util.*;

interface CreditCard {
    public void giveBankDetails();
    public String getCreditCard();
}

class BankDetails{
    private String bankName;
    private String accHolderName;
    private long accNumber;

    public String getBankName() {
        return bankName;
    }
    public void setBankName(String bankName) {
        this.bankName = bankName;
    }
    public String getAccHolderName() {
        return accHolderName;
    }
    public void setAccHolderName(String accHolderName) {
        this.accHolderName = accHolderName;
    }
    public long getAccNumber() {
        return accNumber;
    }
    public void setAccNumber(long accNumber) {
        this.accNumber = accNumber;
    }
} // End of the BankDetails class.

class BankCustomer extends BankDetails implements CreditCard {
    public void giveBankDetails(){
        try{
            Scanner sc = new Scanner(System.in);

            System.out.print("Enter the account holder name :");
```

```

String customername= sc.nextLine();

System.out.print("Enter the bank name :");
String bankname= sc.nextLine();

System.out.print("Enter the account number:");
long accno= sc.nextLong();

setAccHolderName(customername);
setAccNumber(accno);
setBankName(bankname);
}catch(Exception e){
    e.printStackTrace();
}
}
@Override
public String getCreditCard() {
    long accno=getAccNumber();
    String accholdername=getAccHolderName();
    String bname=getBankName();

    return ("The Account number "+accno+" of "+accholdername+" in "+bname+
" bank is valid and authenticated for issuing the credit card.");
}
}

public class Classroom2{
    public static void main(String args[]){
        CreditCard targetInterface=new BankCustomer();
        targetInterface.giveBankDetails();
        System.out.print(targetInterface.getCreditCard());
    }
}

```